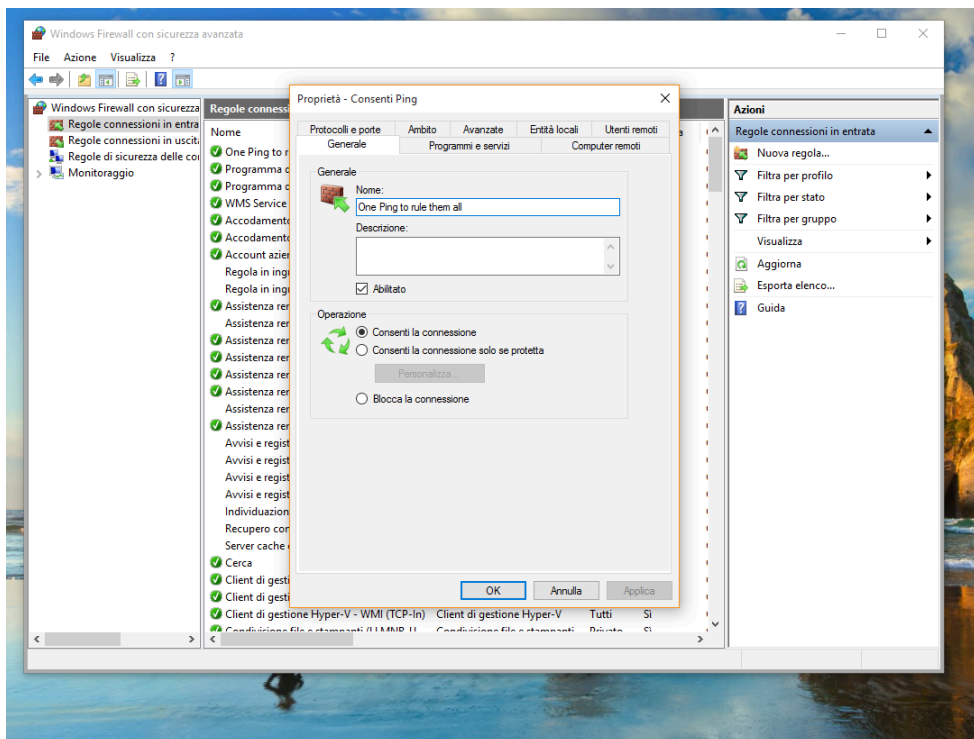
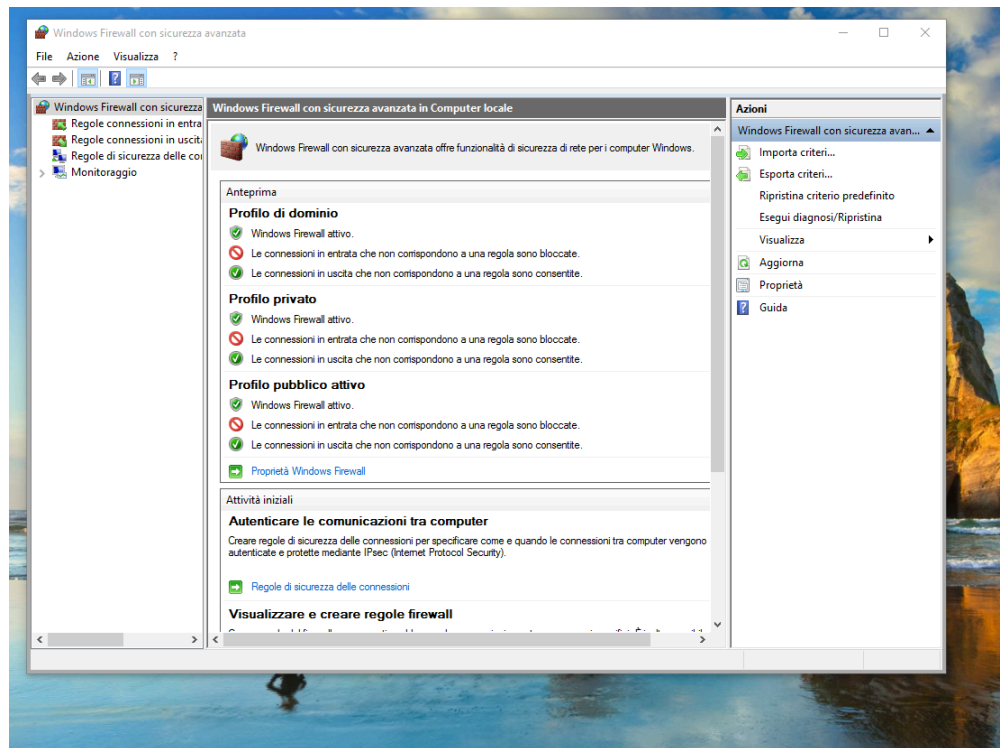


W3D4 - Policy & Packet Capture

Nell'esercizio di oggi andremo a stabilire una policy all'interno del firewall di Windows per permettere il ping proveniente da altre macchine, e poi simuleremo su Kali una serie di servizi che andremo ad interrogare dalla macchina windows, e ne cattureremo i pacchetti da analizzare tramite l'uso di WireShark.

ICMP allow policy su Windows

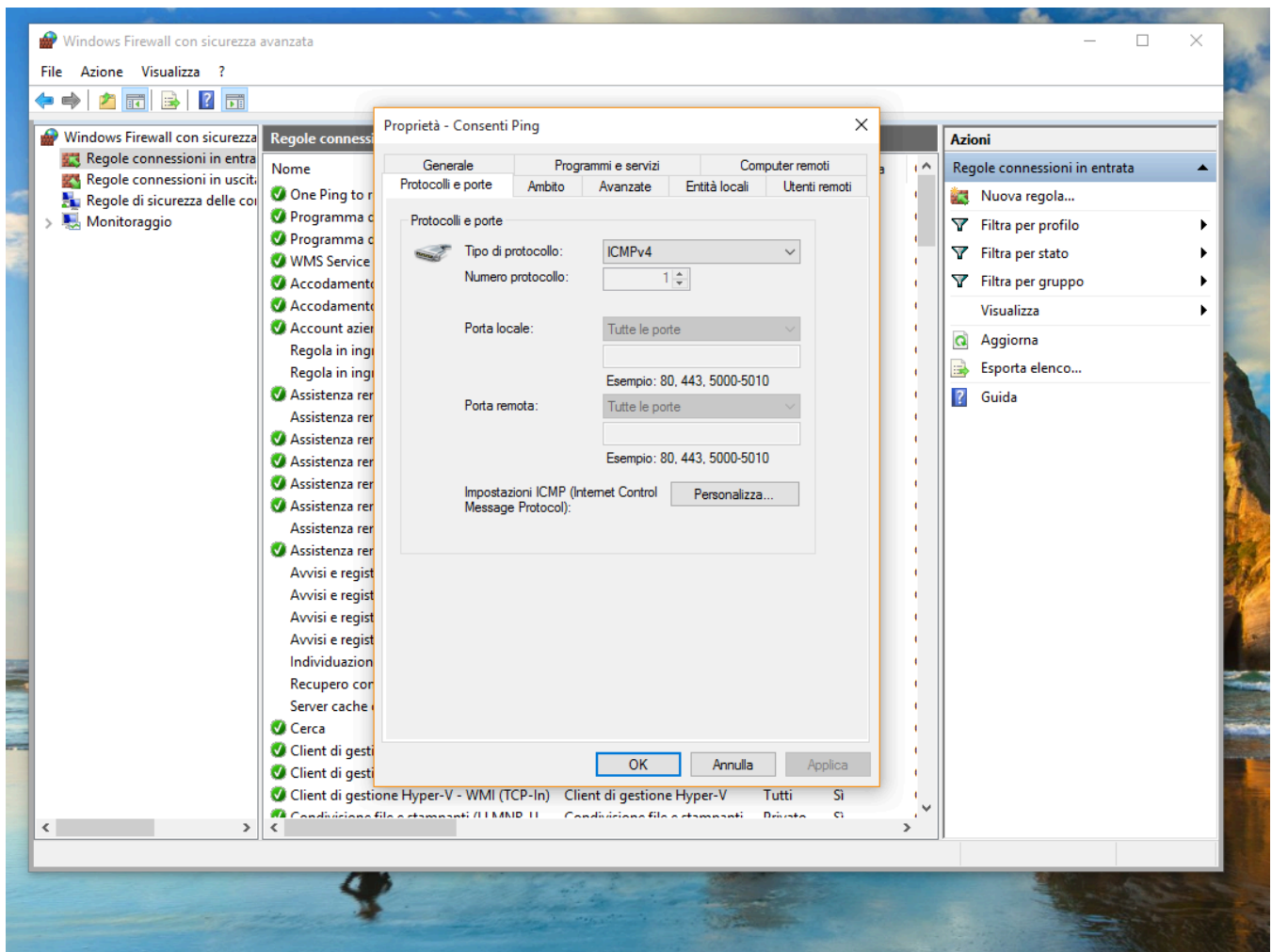
Aviamo l'utility firewall su windows e notiamo subito che le policy sono attive su tutti i profili di dominio. In questa situazione di base tutto il traffico in entrata che non corrisponde ad una regola definita, verrà bloccato.



Per creare la nostra regola dovremo cliccare sulla colonna di sinistra nella sezione Regole connessioni in entrata. Chiameremo la nostra regola **"One ping to rule them all"**.

Come si vede dallo screenshot qui a sinistra, la regola è abilitata e consente le connessioni, quindi **lascierà passare** il traffico in entrata. Le tipologie di traffico permesse le specifichiamo nella scheda

Protocolli e porte, in cui andremo a specificare il protocollo **ICMP** come da screenshot seguente.



Una volta salvata la regola sarà subito attiva, nei due screenshot seguenti si può vedere l'output del programma ping, lanciato dalla nostra Kali verso il computer Windows.

A sinistra si nota che i pacchetti sono andati persi, in realtà sono stati "droppati" silenziosamente, quindi la Kali non sa se siano arrivati a destinazione non avendo ricevuto risposta da Windows. Questo è il comportamento di default quando un pacchetto viene rifiutato.

Nella schermata di destra invece si vede come i pacchetti siano arrivati a destinazione e windows abbia risposto al ping. Questo comportamento è dovuto alla nostra policy appena creata.

```
File Actions Edit View Help
(kali@kali)-[~]
$ ping -c 4 192.168.50.102
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data.
— 192.168.50.102 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3056ms
```

```
File Actions Edit View Help
(kali@kali)-[~]
$ ping -c 4 192.168.50.102
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data.
64 bytes from 192.168.50.102: icmp_seq=1 ttl=128 time=0.533 ms
64 bytes from 192.168.50.102: icmp_seq=2 ttl=128 time=0.410 ms
64 bytes from 192.168.50.102: icmp_seq=3 ttl=128 time=0.343 ms
64 bytes from 192.168.50.102: icmp_seq=4 ttl=128 time=0.418 ms
— 192.168.50.102 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3068ms
rtt min/avg/max/mdev = 0.343/0.426/0.533/0.068 ms
(kali@kali)-[~]
```

Packet capture con Wireshark

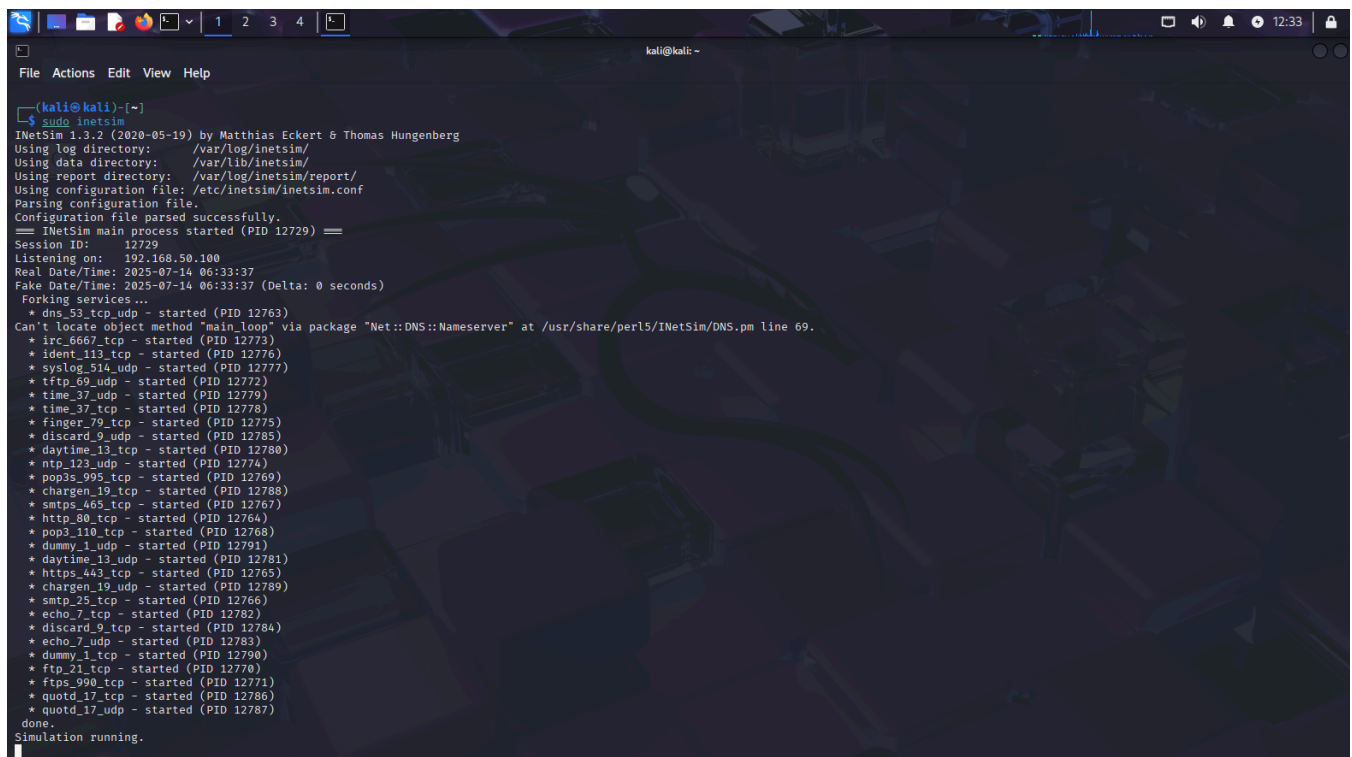
InetSim setup

Procederemo adesso a simulare dei servizi sulla nostra Kali, utilizzando il programma InetSim. Prima di lanciarlo occorre configurarlo in modo da accettare connessioni sulla rete locale, altrimenti di default il programma sarà raggiungibile solo da localhost.

Per configurare inetsim andremo ad editare il file `/etc/inetsim/inetsim.conf`, in particolare il valore di `service_bind_address` inserendo l'indirizzo IP della nostra Kali, come da screenshot a destra.

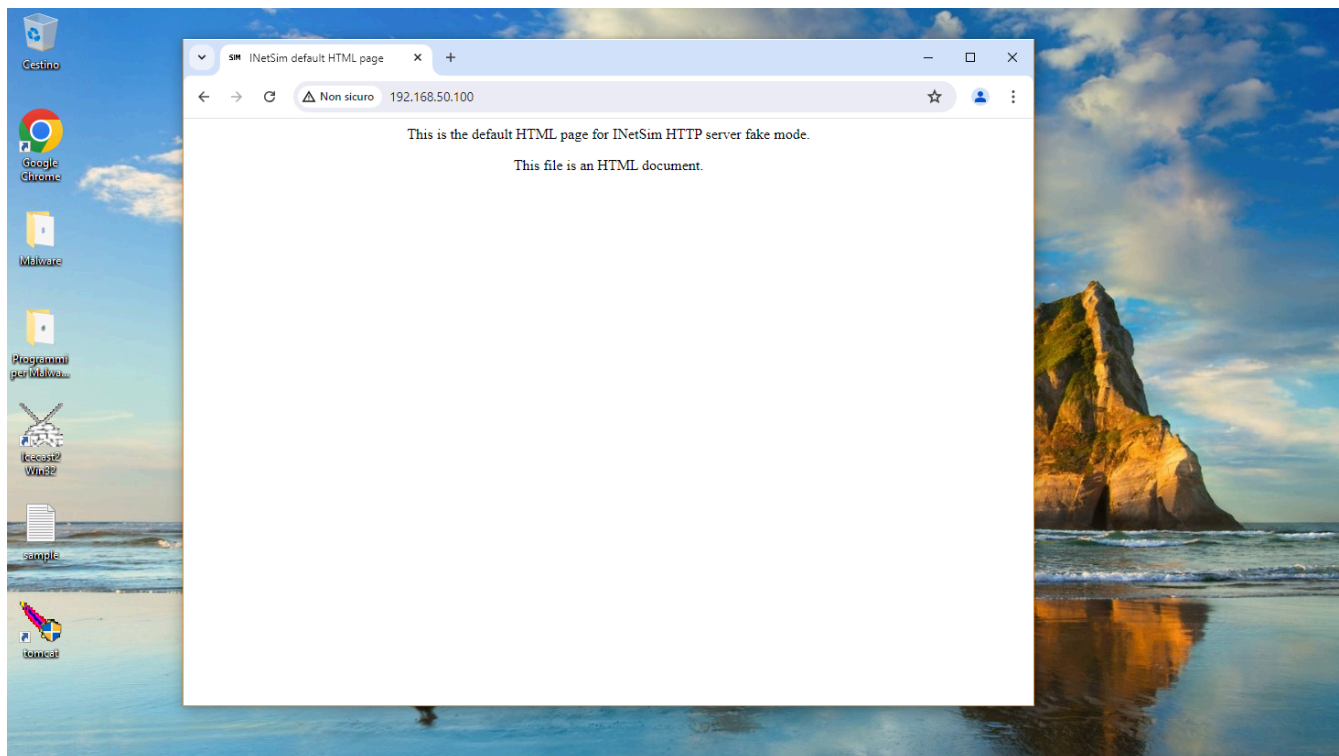
```
#####  
# service_bind_address  
#  
# IP address to bind services to  
#  
# Syntax: service_bind_address <IP address>  
#  
# Default: 127.0.0.1  
#  
#service_bind_address 10.10.10.1  
service_bind_address 192.168.50.100
```

Adesso lanciando il programma vedremo che avvierà tutta una serie di servizi e li metterà in ascolto sull'indirizzo IP da noi assegnato.



```
(kali@kali) ~  
$ sudo inetsim  
InetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg  
Using log directory: /var/log/inetsim/  
Using data directory: /var/lib/inetsim/  
Using report directory: /var/log/inetsim/report/  
Using configuration file: /etc/inetsim/inetsim.conf  
Parsing configuration file.  
Configuration file parsed successfully.  
== InetSim main process started (PID 12729) ==  
Session ID: 12729  
Listening on: 192.168.50.100  
Real Date/Time: 2025-07-14 06:33:37  
Fake Date/Time: 2025-07-14 06:33:37 (Delta: 0 seconds)  
Forking services...  
* dns_53_tcp_udp - started (PID 12763)  
Can't locate object method "main_loop" via package "Net::DNS::Nameserver" at /usr/share/perl5/INetSim/DNS.pm line 69.  
* irc_6667_tcp - started (PID 12773)  
* ident_113_tcp - started (PID 12776)  
* syslog_514_udp - started (PID 12777)  
* tftp_69_udp - started (PID 12772)  
* time_37_udp - started (PID 12779)  
* time_37_tcp - started (PID 12778)  
* finger_79_tcp - started (PID 12775)  
* discard_9_udp - started (PID 12785)  
* daytime_13_tcp - started (PID 12780)  
* ntp_123_udp - started (PID 12774)  
* pop3s_995_tcp - started (PID 12769)  
* chargen_19_tcp - started (PID 12788)  
* smtps_465_tcp - started (PID 12767)  
* http_80_tcp - started (PID 12764)  
* pop3_110_tcp - started (PID 12768)  
* dummy_1_udp - started (PID 12791)  
* daytime_13_udp - started (PID 12781)  
* https_443_tcp - started (PID 12765)  
* chargen_19_udp - started (PID 12789)  
* smtp_25_tcp - started (PID 12766)  
* echo_7_tcp - started (PID 12782)  
* discard_9_tcp - started (PID 12784)  
* echo_7_udp - started (PID 12783)  
* dummy_1_tcp - started (PID 12780)  
* ftp_21_tcp - started (PID 12770)  
* ftps_990_tcp - started (PID 12771)  
* quotd_17_tcp - started (PID 12786)  
* quotd_17_udp - started (PID 12787)  
done.  
Simulation running.
```

Andando ora sulla nostra macchina windows, ci basterà lanciare il browser e puntarlo all'indirizzo IP della Kali per vedere la pagina iniziale del servizio http.



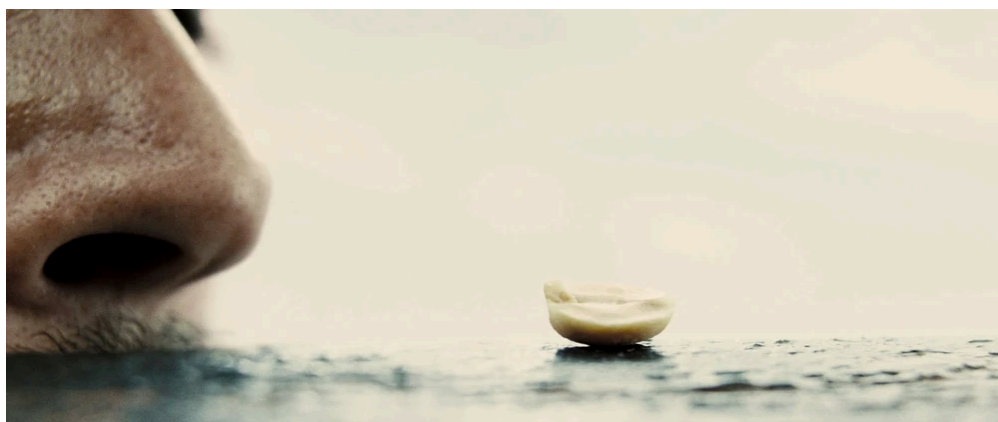
Wireshark

A questo punto abbiamo stabilito del traffico tra le nostre due macchine, possiamo quindi avviare wireshark sulla Kali e procedere a *"sniffare"* i pacchetti in transito tra le due macchine.

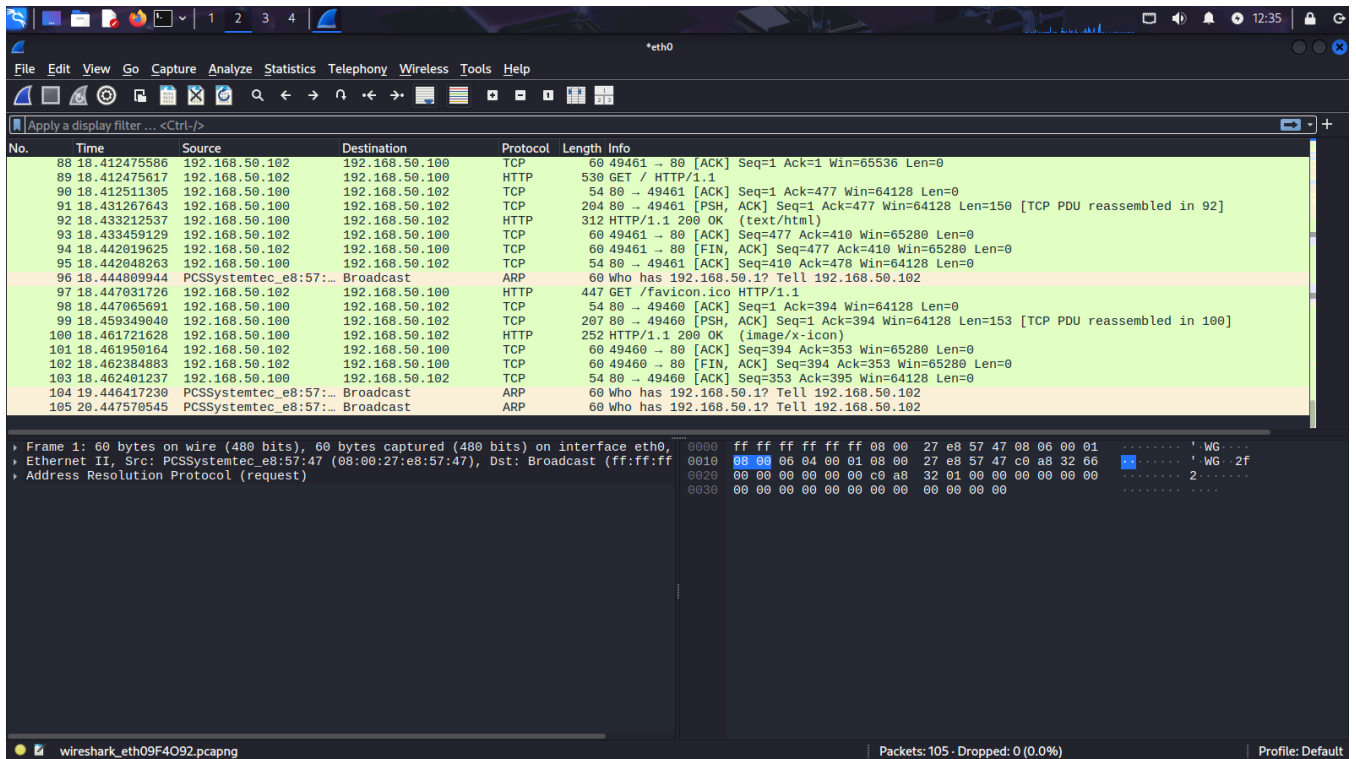
Avviamo wireshark e lo mettiamo in ascolto sull'interfaccia `eth0` visto che sappiamo essere

l'interfaccia che abbiamo impostato per la rete locale, in alternativa avremmo potuto selezionare l'opzione **"any"** per mettere in ascolto su tutte le interfacce disponibili, ma questo avrebbe generato molto più rumore e molti più pacchetti che non ci interessano al momento.

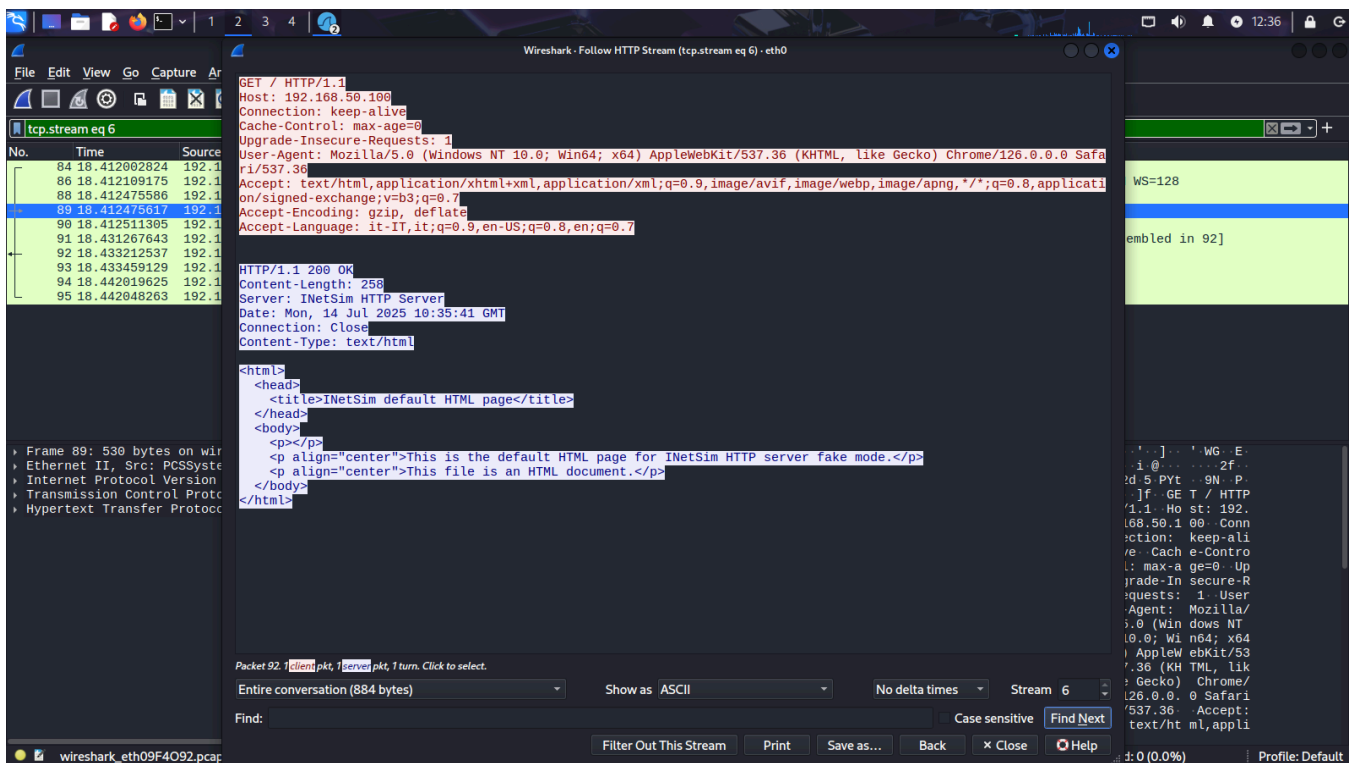
Dopo un refresh della pagina sul browser della macchina Windows, vedremo apparire tutta una serie di pacchetti nella schermata di wireshark, e cliccandoci sopra potremo ispezionarne il contenuto.



Packet/Peanut Sniffing 😊



In particolare, cliccando col tasto destro sul pacchetto con protocollo HTTP (il numero 89 nel nostro screenshot qui sopra), e selezionando l'opzione "follow http stream" potremo visualizzare lo scambio di richieste e risposte tra il client (il browser) in rosso e il server http in blu.



Siccome il traffico è avvenuto su protocollo http, tutte le informazioni sono in chiaro e possono essere visualizzate semplicemente.

Esercizio facoltativo

Utilizzando gli stessi servizi attivati in precedenza con InetSim, possiamo testare il traffico verso altri protocolli, come ad esempio FTP (File Transfer Protocol), quindi andiamo sulla macchina Windows e avviamo da riga di comando l'utilità ftp, puntandola all'IP della nostra Kali.

```
C:\Users\user>ftp 192.168.50.100
C:\Users\user>ftp 192.168.50.100
Connesso a 192.168.50.100.
220 INetSim FTP Service ready.
500 Unknown command.
Utente (192.168.50.100:(none)): kali
331 Please specify the password.
Password:
230 Login successful.
ftp> _
```

Vedremo tutti i pacchetti che le due macchine si scambiano, e anche quì, potremo andare ad analizzarne il contenuto.

The screenshot shows the Wireshark interface with a packet capture on the eth0 interface. The packet list pane displays a series of packets, including a TCP SYN, ACK and several FTP commands. The packet details pane shows the structure of a TCP segment with source port 49465 and destination port 21.

No.	Time	Source	Destination	Protocol	Length	Info
2	1.472852363	192.168.50.102	192.168.50.100	TCP	60	49465 → 21 [SYN, ACK] Seq=0 Win=8192 Len=0 MSS=1460 WS=1 SACK_PERM
3	1.472885537	192.168.50.100	192.168.50.102	TCP	60	21 → 49465 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
4	1.473139810	192.168.50.102	192.168.50.100	TCP	60	49465 → 21 [ACK] Seq=1 Ack=1 Win=8192 Len=0
5	1.477699576	192.168.50.100	192.168.50.102	FTP	86	Response: 220 INetSim FTP Service ready.
6	1.483836651	192.168.50.102	192.168.50.100	FTP	68	Request: OPTS UTF8 ON
7	1.483873614	192.168.50.100	192.168.50.102	TCP	54	21 → 49465 [ACK] Seq=33 Ack=15 Win=64256 Len=0
8	1.485073949	192.168.50.100	192.168.50.102	FTP	76	Response: 500 Unknown command.
10	1.547620252	192.168.50.102	192.168.50.100	TCP	60	49465 → 21 [ACK] Seq=15 Ack=55 Win=8138 Len=0
31	56.919349310	192.168.50.102	192.168.50.100	FTP	65	Request: USER kali
32	56.920597730	192.168.50.100	192.168.50.102	FTP	88	Response: 331 Please specify the password.
33	56.981517799	192.168.50.102	192.168.50.100	TCP	60	49465 → 21 [ACK] Seq=26 Ack=89 Win=8104 Len=0
34	58.482440720	192.168.50.102	192.168.50.100	FTP	65	Request: PASS kali
35	58.483641341	192.168.50.100	192.168.50.102	FTP	77	Response: 230 Login successful.
36	58.545091964	192.168.50.102	192.168.50.100	TCP	60	49465 → 21 [ACK] Seq=37 Ack=112 Win=8081 Len=0

Frame 2: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface eth0, ...
Ethernet II, Src: PCSSystemtec_e8:57:47 (08:00:27:e8:57:47), Dst: PCSSystemtec_d1:f8:11 (08:00:27:f8:11)
Internet Protocol Version 4, Src: 192.168.50.102, Dst: 192.168.50.100
Transmission Control Protocol, Src Port: 49465, Dst Port: 21, Seq: 0, Len: 0

Notiamo subito che username e password sono in chiaro, ma possiamo visualizzare anche i files che vengono trasferiti tra le due macchine, ad esempio provando a scaricare un file da Windows, sar  possibile vedere lo scambio in wireshark.

```
Prompt dei comandi

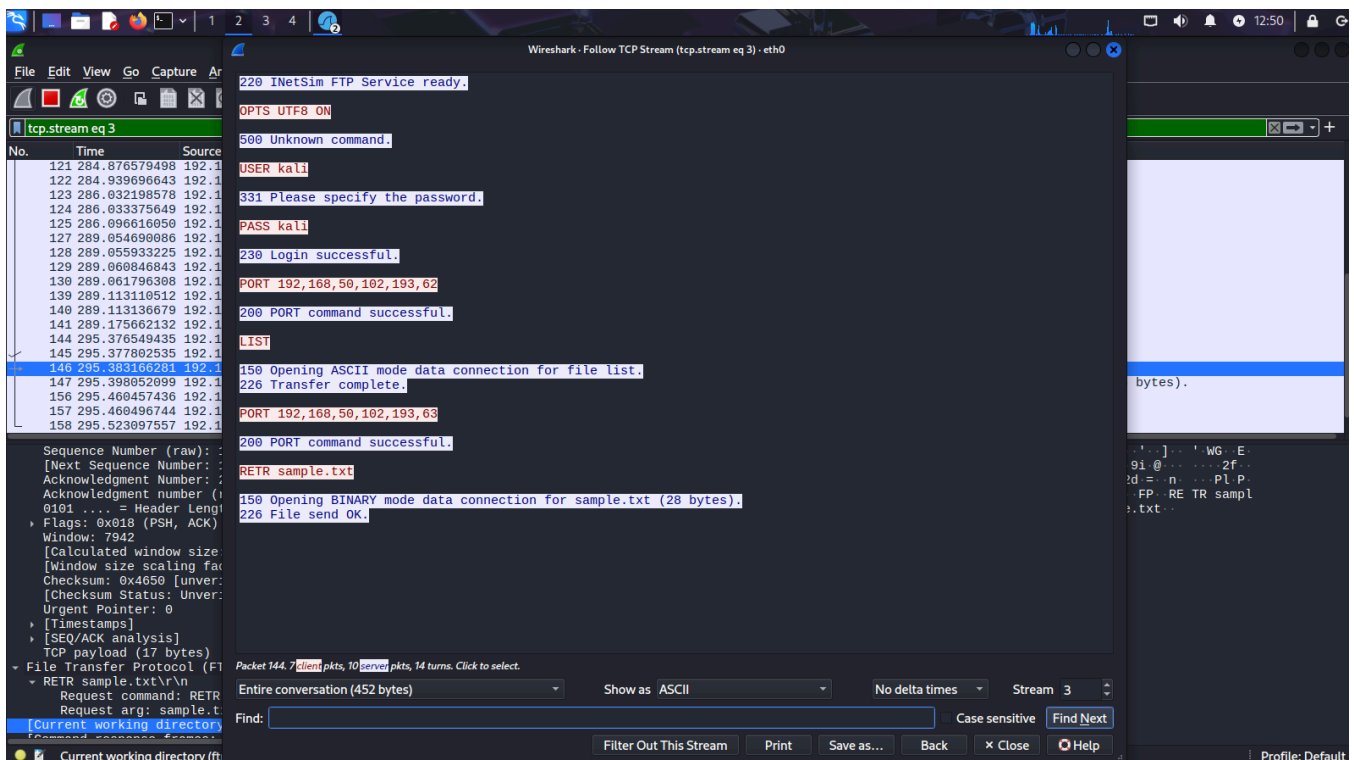
C:\Users\user>ftp 192.168.50.100
Connesso a 192.168.50.100.
220 INetSim FTP Service ready.
500 Unknown command.
Utente (192.168.50.100:(none)): kali
331 Please specify the password.
Password:
230 Login successful.
ftp> dir
200 PORT command successful.
150 Opening ASCII mode data connection for file list.
-rw-rw-rw- 1 1025 1025 28 Jul 11 13:45 sample.txt
226 Transfer complete.
ftp: 70 bytes received in 0.00secondi 70000.00Kbyte/sec)
ftp> get sample.txt
200 PORT command successful.
150 Opening BINARY mode data connection for sample.txt (28 bytes).
226 File send OK.
ftp: 28 bytes received in 0.00secondi 28000.00Kbyte/sec)
ftp> quit
221 Goodbye.

C:\Users\user>
```

Nell'esempio qui a sinistra, abbiamo chiesto al server FTP la lista dei files disponibili con il comando `dir` e poi abbiamo usato il comando `get` per scaricare il file `sample.txt`.

file transfer using FTP

Andando su wireshark e ispezionando lo scambio di pacchetti vediamo tutta la comunicazione in chiaro tra server e client.



Con questo si conclude il nostro esercizio.
Alla prossima.

Daniel MH